

Process	CPU	Private Bytes	Working Set	PID	Description	Company Name
smss.exe	96.52	60 K	228	0		
csrss.exe	0.37	104 K	33,264 K	4		
services.exe	0.37	0 K	0 K	n/a	Hardware Interrupts and DPCs	
svchost.exe	1.148 K	1,408 K	828	328		
svchost.exe	5.052 K	1,614,704 K	3936	1150		
svchost.exe	2.320 K	5,844 K	1056	1160		
svchost.exe	1.704 K	5,424 K	1160	1364		
svchost.exe	6.076 K	11,052 K	1364	1560	Host Process for Windows S...	Microsoft Corporation
svchost.exe	12.452 K	38,316 K	38,316 K	5556		
svchost.exe	29.040 K	39,308 K	5556	5556		
svchost.exe	2.528 K	11,416 K	7884	8220	Sink to receive asynchronou...	Microsoft Corporation
svchost.exe	25.260 K	81,616 K	8220	8220	Microsoft Cross Device Servi...	Microsoft Corporation
svchost.exe	50.096 K	117,586 K	10796	10796		
svchost.exe	39.760 K	115,116 K	12260	12260	Microsoft Edge WebView2	Microsoft Corporation
svchost.exe	2.868 K	15,360 K	12112	12112	Microsoft Edge WebView2	Microsoft Corporation
svchost.exe	12.904 K	41,440 K	12744	12744	Microsoft Edge WebView2	Microsoft Corporation
svchost.exe	34.140 K	53,456 K	12900	12900	Microsoft Edge WebView2	Microsoft Corporation
svchost.exe	8.296 K	21,124 K	12884	12884	Microsoft Edge WebView2	Microsoft Corporation
svchost.exe	87.636 K	72,516 K	13920	13920	Microsoft Edge WebView2	Microsoft Corporation
svchost.exe	83.636 K	128,148 K	11016	11016	Windows Start Experience H...	Microsoft Corporation
svchost.exe	44.652 K	52,032 K	11036	11036		
svchost.exe	7.904 K	49,664 K	11196	11196	Runtime Broker	Microsoft Corporation
svchost.exe	1.808 K	10,464 K	11268	11268	User OOBE Broker	Microsoft Corporation
svchost.exe	6.028 K	28,496 K	11556	11556	WidgetService.exe	Microsoft Corporation
svchost.exe	28.308 K	103,308 K	11020	11020	Windows Start Experience H...	Microsoft Corporation
svchost.exe	11.612 K	51,380 K	9072	9072		
svchost.exe	2.512 K	13,564 K	7544	7544	Runtime Broker	Microsoft Corporation
svchost.exe	149.684 K	238,556 K	7552	7552		
svchost.exe	58.752 K	157,524 K	6680	6680	Microsoft Phone Link	Microsoft Corporation
svchost.exe	11.560 K	47,180 K	2596	2596		
svchost.exe	10.920 K	56,172 K	18152	18152	ISOC	Intel Corporation
svchost.exe	2.516 K	14,652 K	17888	17888	Runtime Broker	Microsoft Corporation
svchost.exe	3.288 K	22,092 K	8020	8020	Runtime Broker	Microsoft Corporation
svchost.exe	4.840 K	32,456 K	18788	18788	Runtime Broker	Microsoft Corporation
svchost.exe	5.404 K	15,136 K	16728	16728		
svchost.exe	9.960 K	32,880 K	12588	12588	Runtime Broker	Microsoft Corporation
svchost.exe	12.452 K	35,684 K	5516	5516	Microsoft OneDrive Co-A...	Microsoft Corporation
svchost.exe	39.424 K	98,516 K	10128	10128	Microsoft OneDrive	Microsoft Corporation
svchost.exe	2.900 K	22,616 K	26228	26228	COM SunGate	Microsoft Corporation
svchost.exe	2.192 K	17,124 K	29944	29944	COM SunGate	Microsoft Corporation
svchost.exe	2.656 K	22,132 K	30512	30512	COM SunGate	Microsoft Corporation
svchost.exe	2.304 K	18,344 K	26160	26160	COM SunGate	Microsoft Corporation
svchost.exe	2.312 K	19,292 K	3292	3292	COM SunGate	Microsoft Corporation
svchost.exe	1.888 K	15,880 K	27968	27968	COM SunGate	Microsoft Corporation
svchost.exe	2.324 K	18,920 K	29550	29550	COM SunGate	Microsoft Corporation
svchost.exe	2.620 K	12,660 K	15640	15640	COM SunGate	Microsoft Corporation
svchost.exe	2.408 K	20,016 K	2340	2340	COM SunGate	Microsoft Corporation
svchost.exe	2.848 K	13,224 K	26020	26020	Windows Defender SmartSe...	Microsoft Corporation
svchost.exe	1.956 K	10,620 K	18728	18728	Runtime Broker	Microsoft Corporation
svchost.exe	10.124 K	20,096 K	1680	1680	Host Process for Windows S...	Microsoft Corporation

есть такая утилита, мне кажется она написана
 Русиновичем. Как мне реализовать подобное через
 питон? К най можно как-то подключиться?

Да. На скриншоте именно **Process Explorer** из **Microsoft Sysinternals**, и вы правильно помните: автор — **Mark Russinovich**. Microsoft сейчас распространяет **Process Explorer**; актуальная версия на официальной странице — 17.13. Microsoft Learn

И здесь есть важный момент: вам не обязательно подключаться к самому **Process Explorer**. Гораздо интереснее сделать собственный аналог на **Python**, обращаясь непосредственно к **Windows API**.

Что именно показывает Process Explorer

В вашем скриншоте видны, например:

- дерево процессов;
- PID;
- CPU;
- Private Bytes;
- Working Set;
- родительский процесс;
- описание;
- издатель;
- службы внутри `svchost.exe` ;
- дочерние процессы;
- отдельные экземпляры `WebView2`;

- системные процессы.

А Process Explorer умеет значительно больше: DLL, handles, потоки, дескрипторы, пользователей, зависимости и т. д. Microsoft прямо указывает, что он показывает открытые процессами handles и загруженные DLL.  Microsoft Learn

1. Можно ли подключиться к Process Explorer?

Технически можно взаимодействовать с его окном/процессом средствами Windows, но это не тот путь, который я рекомендую.

У Process Explorer нет нормального публичного Python API типа:

Python

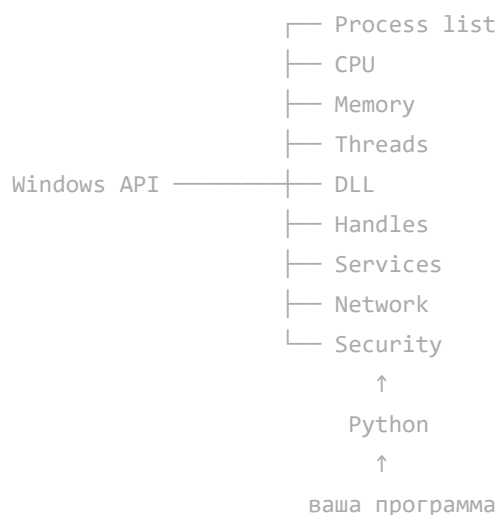
Run

И делать:

Python → Process Explorer → получить данные

не имеет большого смысла.

Лучше:



То есть Process Explorer — хороший эталон того, что надо отображать, а Windows API — источник данных.

2. Для Python уже есть очень хороший первый уровень

Самый простой вариант — `psutil`.

Например:

```
import psutil

for process in psutil.process_iter(
    [
        'pid',
        'ppid',
        'name',
        'exe',
        'username',
        'status',
        'cpu_percent',
        'memory_info',
    ]
):
    try:
        info = process.info

        print(
            info['pid'],
            info['ppid'],
            info['name'],
            info['cpu_percent'],
            info['memory_info'],
        )

    except (psutil.NoSuchProcess, psutil.AccessDenied):
        continue
```

Это уже даст основу для таблицы примерно такого типа:

PID	PPID	Process	CPU	Memory

188	4	Secure System	0.0	...
228	4	Registry	0.0	...
1056	1160	csrss.exe	0.3	...
1384	1160	services.exe	0.0	...
1560	1384	svchost.exe	0.0	...
12260	10792	msedgewebview2.exe	0.1	...

3. Но дерево процессов тоже легко сделать

У Process Explorer очень характерное дерево:

```
System
├─ smss.exe
├─ csrss.exe
├─ wininit.exe
│   └─ services.exe
│       └─ svchost.exe
│           └─ ...
│               └─ ...
└─ ...
```

Поскольку Windows предоставляет PID и PPID, дерево строится элементарно:

```
from collections import defaultdict

processes = list(
    psutil.process_iter(
        ['pid', 'ppid', 'name']
    )
)

children = defaultdict(list)

for process in processes:
    info = process.info
    children[info['ppid']].append(info['pid'])
    ...
```

После этого можно строить настоящий TreeView.

4. А вот здесь начинается самое интересное

Если вы хотите реально приблизиться к Process Explorer, `psutil` будет недостаточно.

Нужно разделить программу на уровни.

Уровень 1 — обычный Process Manager

Можно получить:

- PID;
- PPID;
- имя;
- путь EXE;

- CPU;
- память;
- статус;
- пользователь;
- время запуска;
- количество потоков;
- количество handles.

Для этого достаточно `psutil` + Win32 API.

Уровень 2 — Windows API

Python может непосредственно обращаться к:

- `kernel32.dll` ;
- `advapi32.dll` ;
- `psapi.dll` ;
- `ntdll.dll` ;
- Service Control Manager;
- Windows Management Instrumentation / CIM.

Например, Windows предоставляет `GetProcessMemoryInfo()` для получения информации об использовании памяти процесса.  Microsoft Learn

A `QueryFullProcessImageName()` позволяет получить полный путь к EXE процесса.



Python может вызывать эти функции через:

Python

Run

или через:

`pywin32`

5. DLL

В Process Explorer можно выбрать процесс и увидеть примерно:

```
explorer.exe
├─ ntdll.dll
├─ kernel32.dll
├─ kernelbase.dll
├─ user32.dll
├─ shell32.dll
├─ ...
```

Windows предоставляет для этого механизмы snapshot/module enumeration. Например, `CreateToolhelp32Snapshot()` позволяет получать сведения о процессах и модулях.  Microsoft Learn

В Python это можно реализовать через `ctypes` или `pywin32`.

6. Handles — вот это уже настоящий Process Explorer

Например:

```
explorer.exe
Handles

File
C:\Users\...\Desktop\...

Mutant
...

Section
...

Event
...

Key
HKCU\...
```

Это намного сложнее.

Windows имеет внутренний API:

```
NtQuerySystemInformation()
```

который позволяет получать, в частности, сведения о процессах и количестве используемых ими handles. Microsoft прямо документирует

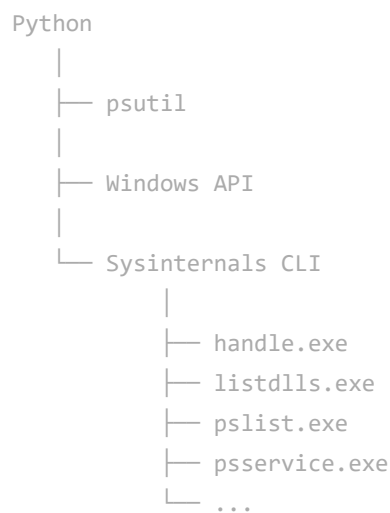
`SystemProcessInformation`.  Microsoft Learn

Но здесь есть важная оговорка: `NtQuerySystemInformation()` — **внутренний API Windows**, и Microsoft предупреждает, что его структура и доступность могут изменяться между версиями Windows.  Microsoft Learn

Поэтому я бы не строил всю программу непосредственно на `NtQuerySystemInformation`.

7. Есть ещё более практичный вариант

Вместо того чтобы самому реализовывать низкоуровневый поиск handles, можно использовать Sysinternals:



Например, официальный `handle` как раз предназначен для поиска открытых handles процессов. Он показывает, какой процесс открыл файл, и тип/имя объекта.

 Algebra Lab

Python может запускать его:

```
import subprocess

result = subprocess.run(
    [
        'handle.exe',
        '-p',
        'explorer.exe',
    ],
    capture_output=True,
    text=True,
    check=False,
```

)

Python

Run

И уже разбирать результат.

Это **намного проще**, чем сразу писать собственную реализацию enumeration всех handles через NT API.

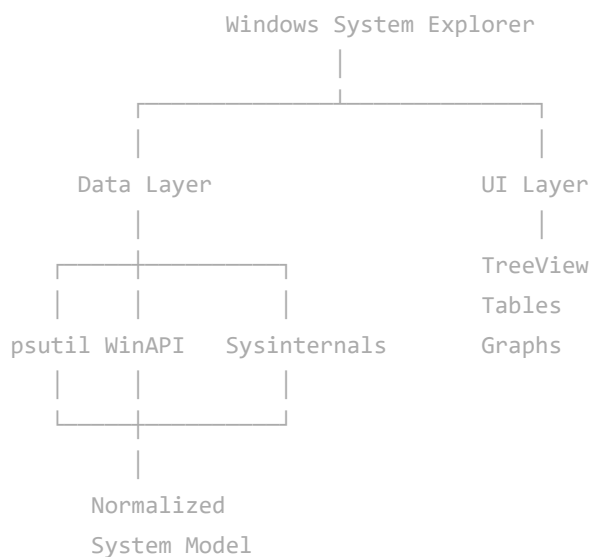
8. И я бы для вашего проекта сделал ещё интереснее

Учитывая, что вы сейчас как раз проектируете приложения для администрирования Windows, я бы не делал просто копию Process Explorer.

Я бы сделал отдельный компонент:

Windows System Explorer

с архитектурой:



И главное — не привязывать UI непосредственно к psutil или WinAPI.

Например:

```
@dataclass
class ProcessInfo:
    pid: int
    ppid: int
    name: str
    executable: str | None
```

```
username: str | None
cpu_percent: float
memory_private: int
memory_working_set: int
thread_count: int
...
```

Python

Run

Тогда UI вообще не знает, откуда пришла информация.

9. И можно сделать практически полный аналог

Например, вкладки:



При выборе процесса:

Process: msedgewebview2.exe

PID: 12260

CPU	0.2 %
Private Bytes	39.7 MB
Working Set	115.3 MB
Threads	18
Handles	342
User	...
Path	C:\Program Files\...
Company	Microsoft Corporation
Signature	Microsoft Corporation

А ниже:

[General]
[Threads]
[Modules]
[Handles]
[Environment]
[Security]
[Performance]
[Network]

И самое важное для вашей задачи

Да, Python здесь вполне подходит.

Причём я бы использовал не один инструмент, а комбинацию:

Задача	Технология
Список процессов	psutil
CPU/RAM	psutil + WinAPI
PID/PPID	psutil / WinAPI
Пути EXE	WinAPI
Пользователь	WinAPI / pywin32
Сервисы	WinAPI / WMI/CIM
DLL	ToolHelp32 / WinAPI
Threads	WinAPI
Handles	Sysinternals handle.exe или NT API
Цифровая подпись	WinTrust / PowerShell
Сетевые соединения	Windows IP Helper API / psutil
События	Windows Event Log API
UI	Qt/PySide
Графики	Qt + matplotlib

Задача

Технология

Администрирование

WinAPI + UAC

И Process Explorer я бы оставил установленным как эталон для проверки результатов нашего собственного движка.

Причём это хорошо стыкуется с вашей предыдущей идеей приложения, которое должно читать **все системные журналы**: можно сделать не два независимых приложения, а целый **Windows System Diagnostics / Control Center**, где Process Explorer-подобный модуль будет одним из компонентов. Process Monitor, например, идёт ещё глубже и отслеживает в реальном времени файловую систему, Registry и активность процессов/потоков.  Algebra Lab

Если хотите именно **повторить функциональность Process Explorer максимально близко**, я бы начал с `psutil` + `PySide6`, а затем постепенно добавлял WinAPI-модули. Это позволит получить рабочее приложение уже на первом этапе, не погружаясь сразу в undocumented NT internals.



Секунду, давай поговорим о winapi модулях

Да. Если мы хотим сделать свой Process Explorer-подобный движок, то я бы вообще пока отложил `psutil` и отдельно разобрал именно **Windows API** как слой получения системной информации.

Здесь полезно разделить API на несколько групп:

1. Процессы и память

Основные DLL:

```
kernel32.dll
psapi.dll
ntdll.dll
```

Через них можно получать:

- список процессов;
- PID / PPID;
- полный путь EXE;

- время запуска;
- память процесса;
- Working Set;
- Private Bytes;
- количество потоков;
- приоритет;
- affinity;
- состояние процесса;
- архитектуру процесса;
- токены безопасности.

Например:

```
OpenProcess()  
QueryFullProcessImageNameW()  
GetProcessMemoryInfo()  
GetProcessTimes()  
GetProcessHandleCount()
```

2. Потоки

Здесь уже можно приближаться к тому, что показывает Process Explorer:

```
Process  
├─ Threads  
│   ├── TID 1234  
│   ├── TID 1235  
│   ├── TID 1236  
│   └─ ...
```

Интересны:

```
OpenThread()  
GetThreadTimes()  
GetThreadContext()  
SuspendThread()  
ResumeThread()
```

Но некоторые операции требуют повышенных прав.

3. DLL / Modules

Очень интересная часть.

Например:

```
explorer.exe
├─ ntdll.dll
├─ kernel32.dll
├─ kernelbase.dll
├─ user32.dll
├─ shell32.dll
└─ ...
```

Можно использовать Tool Help API:

```
CreateToolhelp32Snapshot()
Module32FirstW()
Module32NextW()
```

Это позволяет получить:

- имя DLL;
- полный путь;
- base address;
- размер модуля;
- имя модуля.

То есть **DLL Tree Process Explorer** вполне реально воспроизвести через обычный документированный WinAPI.

4. Handles

Вот здесь начинается самая интересная часть.

Windows API не предоставляет такого удобного:

```
GetProcessHandles(pid)
```

для обычного приложения.

Есть несколько подходов:

```
NtQuerySystemInformation()
NtQueryObject()
DuplicateHandle()
```

То есть:

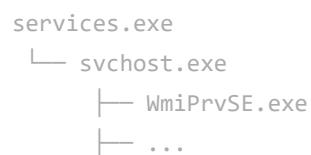


Это уже **NT Native API**, а не обычный Win32 API.

И именно здесь Process Explorer становится существенно сложнее обычного Task Manager.

5. Services

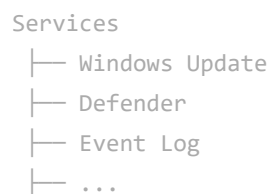
Для вашего случая это особенно интересно, потому что на скриншоте видно:



Windows Service Control Manager предоставляет API:

```
OpenSCManager()
EnumServicesStatusEx()
OpenService()
QueryServiceStatusEx()
```

Через него можно построить:



и связать службу с процессом.

6. Registry

Отдельный WinAPI слой:

```
advapi32.dll
```

Например:

```
RegOpenKeyExW()  
RegQueryValueExW()  
RegEnumKeyExW()
```

Это уже позволяет вашему приложению видеть:

```
HKLM  
├─ SOFTWARE  
├─ SYSTEM  
└─ ...
```

```
HKCU  
├─ Software  
└─ ...
```

И это может пригодиться не только Process Explorer-модулю, но и тому **Windows System Control Center**, который вы сейчас проектируете.

7. Event Log

Здесь тоже advapi32.dll / Windows Event Log API.

Можно работать с:

```
Application  
System  
Security  
Setup  
PowerShell  
Windows Defender  
...
```

и получать:

```
Event ID  
Provider  
Level  
Time  
Message  
Process ID
```

Thread ID
User

Это уже напрямую пересекается с вашим вторым приложением для чтения **всех системных журналов**.

8. Сеть

Для Process Explorer-подобного инструмента я бы отдельно сделал:

iphlpapi.dll

Там есть API для TCP/UDP таблиц.

Можно получить:

```
PID 1234
TCP
0.0.0.0:52341
  ↓
142.250.x.x:443
ESTABLISHED
```

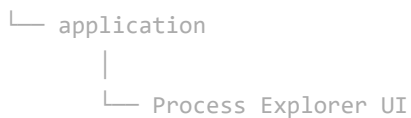
и связать соединение непосредственно с процессом.

Это значительно интереснее простого netstat .

А Python здесь выступает только как оболочка

Архитектурно я бы сделал примерно так:

```
Python
|
|— winapi/
|   |— process.py
|   |— memory.py
|   |— thread.py
|   |— module.py
|   |— handle.py
|   |— service.py
|   |— registry.py
|   |— eventlog.py
|   |— network.py
|   |— security.py
|
```



А внутри, например, `process.py` :

```
import ctypes
```

Python

Run

И дальше мы явно описываем Windows API через `ctypes` .

Но есть ещё один принципиальный вопрос

Для такого проекта я бы разделил WinAPI на три класса:

A. Documented Win32 API

↓

стабильный

рекомендуется Microsoft

B. Native NT API

↓

Nt*

низкоуровневый

часть Process Explorer-функциональности

C. Undocumented structures

↓

SYSTEM_PROCESS_INFORMATION

SYSTEM_HANDLE_INFORMATION

и т.п.

↓

максимально мощно,

но зависит от версии Windows

И вот C — это тот уровень, который стоит отдельно обсудить, потому что именно там появляется возможность получать данные, которых нет в нормальном Win32 API.

Если наша цель — не просто «список процессов на Python», а полноценный Windows Diagnostic Engine, я бы сначала разобрал именно границу Win32 API → NT Native API → undocumented structures. Это определит архитектуру всего приложения.



Так, так, так. Давай сейчас мы соберем большой список, максимально охватывающий все возможности! Собери максимально возможное количество проверок, которые нам могут быть доступны как по документированным, так и по undocumented возможностям

Да. Тогда предлагаю не ограничиваться Process Explorer. Если мы строим собственный Windows Diagnostic / System Explorer engine, лучше сразу составить максимально широкий каталог источников данных и проверок, а уже потом решить, какие из них реализовывать.

И я бы действительно включил три слоя:

1. Documented Win32 / Windows API — основной стабильный слой.
2. WMI/CIM, ETW, Performance Counters, Event Log, SetupAPI и другие штатные механизмы.
3. NT Native / undocumented — nt*, внутренние структуры, системные information classes и т. п.

Microsoft сама относит к диагностическому API процессы, Event Logging, ETW, Performance Counters, Process Snapshotting, PSAPI, WER, Wait Chain Traversal и другие механизмы.  Microsoft Lea... +1

1. Process Intelligence — процессы

Documented

Получаем для каждого процесса:

- PID
- PPID
- имя процесса
- полный путь EXE
- командную строку
- текущую директорию
- пользователя
- SID пользователя
- Session ID

- Integrity Level
- время запуска
- время завершения
- CPU time
- CPU %
- kernel time
- user time
- Working Set
- Peak Working Set
- Private Bytes
- Virtual Size
- Peak Virtual Size
- Pagefile Usage
- Peak Pagefile Usage
- Paged Pool
- Nonpaged Pool
- количество handles
- количество threads
- priority
- base priority
- affinity
- processor affinity mask
- NUMA node
- IO counters
- read operations
- write operations
- read bytes
- write bytes
- network-related activity через соответствующие API/ETW
- архитектуру x86/x64/ARM64
- WOW64
- executable version
- Company Name
- Product Name
- File Description
- Copyright
- цифровую подпись
- сертификат
- hash файла

Для базовой информации Windows уже предоставляет Process Status API и другие документированные функции.

NT / undocumented

Через `NtQuerySystemInformation(SystemProcessInformation)` можно получить массив процессов вместе с потоками, handle count, memory statistics и другими полями внутренней структуры. Microsoft отдельно предупреждает, что этот API и возвращаемые структуры могут изменяться между версиями Windows.  Microsoft Learn

То есть можно добавить:

- sequence number процесса;
- обнаружение PID reuse;
- kernel/user times;
- thread states;
- wait reasons;
- start addresses потоков;
- внутренние memory counters;
- внутренние process flags;
- session information;
- дополнительные process information classes.

Причём в новых Windows 11 появился `SystemBasicProcessInformation`, который Microsoft рекомендует использовать вместо полного `SystemProcessInformation`, когда достаточно базовой информации.  Microsoft Learn

2. Process Tree

Строим настоящее дерево:

```
System
├─ smss.exe
├─ csrss.exe
├─ wininit.exe
│   └─ services.exe
│       └─ svchost.exe
│           └─ ...
└─ explorer.exe
    └─ ...
        └─ ...
```

Проверки:

- отсутствующий parent;

- parent PID reused;
- процесс без ожидаемого parent;
- неожиданный child;
- изменение parent;
- orphan process;
- процессы, запущенные из Temp;
- процессы из пользовательских каталогов;
- процессы из network share;
- процессы из removable media;
- несколько одинаковых процессов;
- необычное дерево запуска.

3. Threads

Для каждого процесса:

- TID;
- PID;
- start address;
- thread state;
- wait reason;
- base priority;
- dynamic priority;
- CPU time;
- kernel time;
- user time;
- affinity;
- ideal processor;
- processor number;
- thread creation time;
- suspended state;
- stack information — при наличии соответствующих механизмов/прав;
- owning process.

Через NT-информацию можно получить thread state и wait reason;

SYSTEM_PROCESS_INFORMATION сопровождается массивами SYSTEM_THREAD_INFORMATION .



Проверки

- thread с необычным start address;
- thread вне ожидаемого module range;

- thread consuming abnormal CPU;
- suspended threads;
- threads waiting unusually long;
- threads belonging to terminated/reused PID;
- anomalous thread start modules.

4. DLL / Modules

Для каждого процесса:

Process

```
|— ntdll.dll  
|— kernel32.dll  
|— kernelbase.dll  
|— user32.dll  
|— ...
```

Проверяем:

- module name;
- full path;
- base address;
- image size;
- entry point;
- version;
- company;
- signature;
- certificate;
- hash;
- loaded time;
- architecture;
- memory permissions.

Аномалии

- unsigned DLL;
- DLL из %TEMP% ;
- DLL из %APPDATA% ;
- DLL из Downloads;
- DLL с duplicate name;
- DLL loaded from unusual location;
- DLL loaded from network share;

- module without valid image backing;
- executable memory not associated with normal image.

5. Handles

Вот это уже практически отдельный большой subsystem.

Проверяем:

```
File
Directory
Key
Event
Mutex
Semaphore
Section
Process
Thread
Token
Job
ALPC
Pipe
Socket-related objects
Timer
Desktop
WindowStation
```

Для каждого:

- owning PID;
- handle value;
- object type;
- object name;
- access mask;
- attributes;
- referenced object;
- duplicated handle;
- inheritance;
- protection.

Аномалии

- процесс держит файл, который неожиданно заблокирован;
- handle на чувствительный файл;
- handle на другой процесс;

- handle на другой thread;
- подозрительные cross-process handles;
- excessive handle count;
- handle leaks;
- unusual object types.

Для этой области NT Native API становится особенно важным.

6. Windows Kernel Objects

Отдельный каталог:

```
Process
Thread
File
Section
Event
Mutant
Semaphore
Token
Job
Key
IoCompletion
ALPC Port
Timer
Directory
SymbolicLink
```

Можно строить Object Explorer:

```
\Device
\Driver
\BaseNamedObjects
\Sessions
\RPC Control
\KnownDlls
\Registry
```

Это уже гораздо ближе к внутренностям Windows, чем обычный Task Manager.

7. Memory

Очень большой блок.

На уровне системы

- total RAM;
- available RAM;
- committed memory;
- commit limit;
- cached memory;
- standby;
- modified;
- zeroed;
- free;
- kernel memory;
- paged pool;
- nonpaged pool;
- page faults;
- hard faults;
- memory compression;
- swap/pagefile;
- NUMA.

На уровне процесса

- Working Set;
- Private Bytes;
- Virtual Size;
- Commit;
- page faults;
- working-set transitions;
- locked memory;
- executable memory;
- writable executable memory.

Проверки

RWX memory
RX anonymous memory
executable private pages
abnormal commit
memory leak
working set explosion
page fault storm
high hard-fault rate
nonpaged pool growth
paged pool growth

8. Virtual Memory Maps

Для процесса:

```
00000000-...  
IMAGE  
MAPPED  
PRIVATE  
HEAP  
STACK  
DLL
```

Проверяем:

- address;
- size;
- state;
- protection;
- type;
- allocation base;
- allocation protection.

Это особенно интересно для диагностики:

- injected code;
- executable private memory;
- manually mapped images;
- suspicious memory regions.

9. Services

Через Service Control Manager:

- service name;
- display name;
- description;
- executable;
- command line;
- service type;
- start type;
- current state;
- PID;
- dependencies;
- dependents;

- account;
- failure actions;
- triggers;
- delayed start;
- recovery configuration.

Проверки

- service without valid executable;
- unsigned service executable;
- service from user directory;
- service from Temp;
- service running as SYSTEM;
- unusual service account;
- orphan service;
- service executable mismatch;
- service path quoting problem;
- service with suspicious arguments.

10. Drivers

Очень важный раздел.

Проверяем:

- kernel drivers;
- loaded .sys ;
- driver path;
- service association;
- start type;
- load order;
- signer;
- certificate;
- hash;
- version;
- company;
- driver state.

Аномалии

- unsigned driver;
- test-signed driver;

- driver from unusual path;
- recently installed driver;
- driver with invalid signature;
- suspicious kernel module.

11. Kernel / NT System

Здесь можно использовать `NtQuerySystemInformation` .

Интересны:

- system process information;
- processor performance;
- system handles;
- loaded modules;
- code integrity;
- kernel debugger state;
- boot environment;
- object information;
- page information;
- lookaside information;
- system performance;
- processor topology;
- session information;
- security-related system information.

Например, Microsoft документирует `SystemCodeIntegrityInformation` , позволяющий получить состояние kernel/user-mode code integrity, включая enforcement и test-signing flags.  Microsoft Learn

Но этот слой надо проектировать как **optional backend**, потому что Microsoft прямо предупреждает о нестабильности `NtQuerySystemInformation` .  Microsoft Learn

12. Registry

Полный Registry Explorer:

HKLM
HKCU
HKCR

HKU
HKCC

Проверяем:

- keys;
- values;
- types;
- timestamps;
- security descriptors;
- ACL;
- owner;
- inherited permissions;
- symbolic links.

Особенно интересны

Run
RunOnce
Services
Winlogon
Shell
Userinit
Image File Execution Options
App Paths
Known DLLs
Policies
Windows Defender
Firewall
Explorer
Startup
Uninstall
Installed Components

13. Autoruns

Можно собрать **собственный Autoruns**.

Проверять:

- Registry Run;
- Startup folders;
- Services;
- Drivers;
- Scheduled Tasks;

- Winlogon;
- Shell extensions;
- Explorer extensions;
- Browser extensions;
- Applnit;
- IFEO;
- WMI permanent subscriptions;
- COM;
- DLL search paths;
- boot execute;
- logon scripts.

И получить:

```
Entry
Location
Command
Executable
User
Signer
Hash
Enabled
Risk
```

14. Scheduled Tasks

Проверяем:

- task name;
- author;
- description;
- executable;
- arguments;
- trigger;
- user;
- principal;
- run level;
- hidden;
- enabled;
- last run;
- next run;
- result;

- repetition;
- boot trigger;
- logon trigger;
- idle trigger.

Аномалии

- task launching from Temp;
- task launching PowerShell;
- hidden task;
- SYSTEM task from user directory;
- unusual trigger;
- unsigned executable.

15. Windows Event Log

И здесь ваш второй проект напрямую подключается.

Источники:

```
Application
System
Security
Setup
PowerShell
Defender
TaskScheduler
WMI
BITS
DNS Client
Firewall
Kernel
Service Control Manager
...
```

Проверяем:

- Event ID;
- provider;
- timestamp;
- level;
- opcode;
- task;
- process ID;
- thread ID;

- user;
- SID;
- computer;
- XML;
- raw event data.

16. ETW — огромный источник

Вот здесь мы можем получить **реально потоковую телеметрию Windows**.

Microsoft перечисляет системные ETW providers, среди которых:

```
System Process
System Thread
System IO
System Memory
System Registry
System Scheduler
System CPU
System Network
System Power
System Interrupt
System Object
System ALPC
System Hypervisor
System Config
```

и другие.  [GitHub](#)

Например, System Process Provider позволяет отслеживать:

- process lifetime;
- image load;
- thread events;
- process counters;
- worker threads;
- job events.  [GitHub](#)

Это позволяет сделать:

```
Real-time System Monitor
```

а не просто периодически опрашивать систему.

17. Performance Counters

Отдельный огромный слой:

CPU
Memory
Disk
Network
Process
Thread
Cache
Paging
TCP
HTTP
GPU
Services
SQL/IIS/...

Windows Performance Counters специально предназначены для диагностического сбора данных и доступны через PDH, PerfLib V2 и WMI.  Microsoft Learn

18. CPU

Можно получать:

- total utilization;
- per-core utilization;
- user/kernel;
- interrupt time;
- DPC time;
- idle;
- frequency;
- base frequency;
- turbo;
- processor topology;
- logical processors;
- physical cores;
- packages;
- NUMA;
- affinity;
- processor groups.

Диагностика

CPU saturation
single-core saturation
DPC storm
ISR storm
CPU frequency throttling
processor imbalance
NUMA imbalance

19. Interrupts / DPC

Через ETW и соответствующие системные counters:

- ISR;
- DPC;
- ISR duration;
- DPC duration;
- interrupt frequency;
- CPU consuming interrupts.

Это очень полезно для поиска:

driver causing high CPU
network driver problem
storage driver problem
USB driver problem

20. Storage

Для дисков:

- physical disks;
- logical disks;
- partitions;
- volumes;
- filesystem;
- mount points;
- sector size;
- cluster size;
- capacity;
- free space;
- read/write activity;
- queue length;

- latency;
- IOPS.

21. Storage health

Дополнительно:

- SMART;
- NVMe health;
- temperature;
- percentage used;
- media errors;
- critical warnings;
- firmware;
- serial;
- model;
- controller.

Через Windows storage IOCTL можно получать гораздо больше, чем показывает обычный Explorer.

22. File System

Можно исследовать:

- open files;
- locks;
- file handles;
- filesystem;
- ACL;
- owner;
- attributes;
- alternate streams;
- reparse points;
- junctions;
- symbolic links;
- file IDs;
- timestamps;
- USN Journal.

23. USN Journal

Отдельная очень мощная возможность.

Можно отслеживать:

```
file created
file deleted
file renamed
file modified
security changed
attributes changed
```

Это позволяет построить:

```
File Activity Monitor
```

почти в реальном времени.

24. Network

Через IP Helper / Winsock / ETW:

- interfaces;
- IPv4;
- IPv6;
- TCP listeners;
- TCP connections;
- UDP endpoints;
- owning PID;
- local address;
- remote address;
- state;
- ports;
- routing table;
- ARP/neighbor table.

Например:

```
PID 1234
TCP
192.168.1.10:52341
→
```

25. Network diagnostics

Дополнительно:

- DNS;
- DNS cache;
- DHCP;
- routes;
- gateways;
- adapters;
- Wi-Fi;
- signal;
- link speed;
- network profiles;
- firewall state;
- firewall rules;
- VPN adapters;
- virtual adapters;
- Hyper-V networking;
- WSL networking;
- NAT.

26. Firewall

Полностью исследовать Windows Filtering Platform / Windows Firewall:

- profiles;
- inbound rules;
- outbound rules;
- application rules;
- ports;
- protocols;
- services;
- interfaces;
- users;
- groups;
- enabled/disabled;

- action;
- logging.

Можно сделать:

```
Process → Network Connection → Firewall Rule
```

Это уже очень мощная диагностика.

27. Security Token

Для процесса:

- token;
- user SID;
- groups;
- privileges;
- integrity level;
- elevation;
- UAC;
- token type;
- impersonation level;
- restricted token;
- AppContainer;
- capabilities.

Проверки:

```
Running as SYSTEM
Running elevated
High integrity
Medium integrity
Low integrity
AppContainer
Restricted
Impersonating
```

28. ACL / Security

Для любого объекта:

- owner;
- DACL;

- SACL;
- ACE;
- inherited permissions;
- explicit permissions;
- effective access.

Можно проверять:

```
Who can modify this EXE?  
Who can modify this service?  
Who can modify this Registry key?  
Who can modify this scheduled task?
```

29. UAC

Проверяем:

- UAC enabled;
- consent mode;
- secure desktop;
- administrator approval;
- virtualization;
- elevation state.

30. Code Integrity

Проверяем:

- Secure Boot;
- kernel code integrity;
- user-mode code integrity;
- test signing;
- vulnerable driver blocklist;
- HVCI;
- Memory Integrity;
- WDAC;
- App Control;
- signature validity.

31. TPM

Получаем:

- TPM presence;
- version;
- manufacturer;
- firmware;
- specification version;
- PCR state;
- readiness;
- ownership;
- EK;
- attestation information — в пределах доступного API.

32. Secure Boot

Проверяем:

```
Secure Boot enabled
UEFI
Boot manager
Boot configuration
Boot entries
Boot device
```

33. Windows Boot Configuration

Исследовать BCD:

```
bootmgr
loader
recovery
hypervisor
debug
safe mode
boot options
```

Проверки:

- unusual boot entry;
- debugging enabled;
- test signing;
- recovery changes;
- Safe Mode configuration;

- hypervisor configuration.

34. Hyper-V / Virtualization

Проверяем:

- Hyper-V;
- VBS;
- HVCI;
- Windows Hypervisor;
- virtual machines;
- virtual switches;
- virtual NIC;
- WSL;
- Sandbox;
- containers;
- virtualization extensions.

35. Hardware

Через SetupAPI / PnP / WMI / Device Installation API:

CPU
GPU
RAM
Motherboard
BIOS
UEFI
USB
PCI
PCIe
NVMe
SATA
Audio
Network
Bluetooth
Camera
Monitor
Keyboard
Mouse
Printers

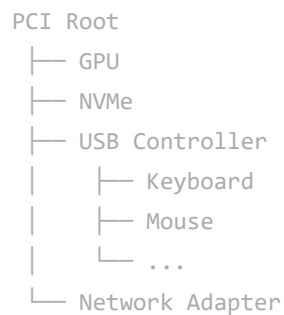
Для каждого:

- Device ID;

- Instance ID;
- Hardware IDs;
- Compatible IDs;
- driver;
- driver version;
- provider;
- signer;
- status;
- problem code;
- location;
- parent;
- children.

36. Device Tree

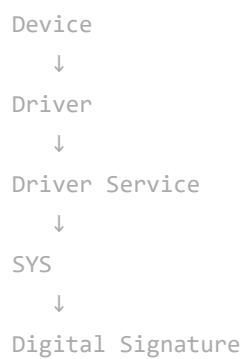
Получаем:



И это уже практически отдельный **Device Explorer**.

37. Drivers ↔ Devices

Очень полезно связать:



И наоборот:

```
driver.sys
  ↓
devices using this driver
```

38. WMI / CIM

WMI даст огромный дополнительный пласт.

Классы:

```
Win32_Process
Win32_Service
Win32_OperatingSystem
Win32_ComputerSystem
Win32_LogicalDisk
Win32_DiskDrive
Win32_NetworkAdapter
Win32_PnPEntity
Win32_PhysicalMemory
Win32_BIOS
Win32_StartupCommand
Win32_ScheduledJob
Win32_UserAccount
Win32_Group
...
```

Но я бы не делал WMI единственным источником. Это скорее ещё один backend.

39. Windows Users

Получаем:

- local users;
- domain users;
- groups;
- SID;
- account state;
- password policy;
- disabled accounts;
- locked accounts;
- last logon;

- group membership;
- privileges.

40. Sessions

Проверяем:

```
Console
RDP
Services
Disconnected
Remote
```

Для каждой:

- Session ID;
- user;
- state;
- logon time;
- client;
- source;
- processes.

41. RDP

Отдельно:

- RDP enabled;
- listening port;
- active sessions;
- disconnected sessions;
- RDP security settings;
- NLA;
- RDP firewall rules;
- RDP event logs.

42. COM / RPC / ALPC

Более низкий уровень:

- COM registrations;

- CLSID;
- AppID;
- COM servers;
- RPC endpoints;
- named pipes;
- ALPC ports.

Это уже интереснейшая область для диагностики зависаний и межпроцессного взаимодействия.

43. Named Pipes

Исследовать:

```
\\.\pipe\
```

и соответствующие kernel objects.

Получаем:

- pipe name;
- server;
- client;
- process;
- access;
- state.

44. Windows Sockets

Для процесса:

```
PID
↓
socket
↓
local endpoint
↓
remote endpoint
↓
TCP state
```

Можно строить Network tab прямо как в Process Explorer/Process Hacker.

45. Wait Chain

Очень полезный механизм диагностики зависаний.

Например:

```
Process A
  ↓ waits for
Thread B
  ↓ waits for
Mutex
  ↓ owned by
Process C
```

Можно обнаруживать:

- deadlocks;
- blocked threads;
- mutex contention;
- resource waits.

Windows имеет отдельный **Wait Chain Traversal API**, который Microsoft относит к диагностическим API.  [GitHub](#)

46. Windows Error Reporting

Можно анализировать:

- crashes;
- application failures;
- faulting module;
- exception code;
- dump availability;
- crash history.

То есть:

```
chrome.exe
↓
crashed
↓
faulting module: xxx.dll
↓
exception: ...
```

47. Application Compatibility

Проверять:

- compatibility settings;
- AppCompat layers;
- shims;
- compatibility database;
- legacy application configuration.

48. Windows Update

Получаем:

- installed updates;
- pending updates;
- failed updates;
- update history;
- reboot required;
- servicing state;
- component store state.

49. Component Store

Диагностика:

```
WinSxS
CBS
DISM
Component Store
```

Проверки:

- component store corruption;
- pending operations;
- servicing state;
- package state.

50. Windows Features

Проверяем:

Hyper-V
WSL
Sandbox
IIS
Telnet
SMB
.NET
OpenSSH
Containers
Virtual Machine Platform
...

51. Windows Defender

Можно анализировать:

- service state;
- engine version;
- signature version;
- last update;
- real-time protection;
- exclusions;
- controlled folder access;
- firewall integration;
- Defender events;
- threat history.

52. PowerShell

Проверяем:

- installed versions;
- execution policy;
- profiles;
- modules;
- logging;
- Script Block Logging;
- transcription;
- constrained language mode;
- PowerShell events.

53. Environment

Для каждого процесса:

```
PATH
TEMP
TMP
USERPROFILE
APPDATA
LOCALAPPDATA
PROGRAMDATA
SYSTEMROOT
WINDIR
USERNAME
USERDOMAIN
PROCESSOR_ARCHITECTURE
...
```

Можно сравнивать environment процесса с системным.

54. File Associations

Исследовать:

```
.exe
.ps1
.py
.txt
.html
...
```

и соответствующие Registry associations.

55. Shell / Explorer

Проверять:

- Explorer extensions;
- shell extensions;
- context menu handlers;
- namespace extensions;
- thumbnail providers;
- preview handlers;
- icon handlers;

- protocol handlers.

56. Browser / WebView

Для вашей системы это особенно актуально.

Можно отдельно отслеживать:

Edge
Chrome
WebView2
CEF
Electron

и:

- processes;
- child processes;
- command line;
- profiles;
- extensions;
- network connections;
- cache;
- subprocess tree.

57. GPU

Получаем:

- adapters;
- VRAM;
- shared GPU memory;
- engine utilization;
- processes using GPU;
- driver;
- driver version;
- temperature — если доступна;
- power state.

58. Audio

- audio devices;
- endpoints;
- default device;
- sessions;
- applications using audio;
- volume;
- mute;
- exclusive mode.

59. Power

Проверяем:

- AC/battery;
- battery health;
- charge;
- discharge;
- power plan;
- sleep;
- hibernate;
- wake timers;
- processor power state.

60. Battery

Для ноутбука:

- capacity;
- design capacity;
- full charge capacity;
- cycle count;
- voltage;
- current;
- temperature;
- health percentage.

61. Time

Проверяем:

- system time;
- UTC;
- timezone;
- NTP;
- synchronization;
- clock drift;
- Windows Time service.

62. Locale

- language;
- locale;
- keyboard layouts;
- code pages;
- regional format;
- timezone.

63. System Configuration

Полный inventory:

```
Windows version
Build
Edition
Architecture
Install date
Boot time
Uptime
Hostname
Domain
Workgroup
CPU
RAM
GPU
Disk
BIOS
UEFI
TPM
Secure Boot
Hyper-V
VBS
```

64. Environment / Configuration Drift

Очень полезный режим.

Создаём snapshot:

```
System Snapshot #1
```

через неделю:

```
System Snapshot #2
```

и сравниваем:

```
+ new service  
+ new driver  
+ new startup entry  
+ new scheduled task  
+ new DLL  
+ registry changed  
+ firewall rule changed  
+ new user  
+ new device  
+ Windows update
```

Это уже превращает программу из Process Explorer в **Windows Forensic / Diagnostic System**.

65. Real-Time Change Detection

Через ETW + Registry notifications + filesystem notifications:

```
Process created  
Process terminated  
Thread created  
DLL loaded  
DLL unloaded  
File created  
File modified  
File deleted  
Registry changed  
Service changed  
Device connected  
Device removed  
Network connection changed
```

User logged in
User logged out

ETW особенно хорошо подходит для этой задачи; Windows имеет отдельные system providers для process, IO, memory, registry, scheduler, network, power, object и других подсистем.  [GitHub](#)

66. Kernel Events

Если пойти совсем глубоко:

Process
Thread
Image Load
Context Switch
DPC
ISR
Disk IO
File IO
Registry
TCP/IP
Memory
Page Fault
Hard Fault
Power

Это уже практически область **Windows Performance Analyzer / Process Monitor / xperf**, а не обычного Process Explorer.

67. Crash / Dump Analysis

Можно добавить:

- minidumps;
- full dumps;
- process dumps;
- exception;
- stack;
- modules;
- threads;
- handles;
- memory regions.

68. Application Hang Detection

Проверять:

- UI thread;
- message queue;
- wait chain;
- blocked thread;
- CPU starvation;
- deadlock;
- synchronous IO;
- network timeout.

69. Resource Leak Detection

Для процесса:

```
Handle count ↑  
Thread count ↑  
Private Bytes ↑  
Working Set ↑  
GDI Objects ↑  
USER Objects ↑
```

Можно вести временной график и обнаруживать:

```
memory leak  
handle leak  
thread leak  
GDI leak
```

70. GDI / USER Objects

Для GUI-процессов:

- GDI handles;
- USER handles;
- windows;
- desktops;
- window stations;
- GUI threads.

71. Windows Objects Namespace

Очень низкий уровень:

```
\Device
\Driver
\ObjectTypes
\BaseNamedObjects
\Sessions
\KnownDlls
\RPC Control
\GLOBAL??
\REGISTRY
```

Можно сделать отдельный:

```
Object Explorer
```

72. Security Audit

Проверять события:

```
Logon
Logoff
Process creation
Service installation
Privilege use
Account changes
Policy changes
Object access
```

73. Process Creation Audit

Связывать:

```
Process
↓
Parent
↓
Command line
↓
User
↓
Token
```

↓
Signature
↓
Network
↓
Files
↓
Registry

Получается process provenance graph.

74. Process Graph

Например:

explorer.exe
↓
powershell.exe
↓
python.exe
↓
chrome.exe
↓
network connection

или:

winword.exe
↓
powershell.exe
↓
cmd.exe

И программа может помечать необычные цепочки.

75. File → Process

Например:

C:\test\database.db
↓
Who has it open?
↓
PID 4520

↓
python.exe

76. Registry Key → Process

То же:

Registry Key
↓
open handles
↓
PID

77. Service → Process → DLL → Network

Именно такой cross-correlation я бы сделал одной из ключевых возможностей:

Service
↓
Process
↓
Threads
↓
Modules
↓
Handles
↓
Files
↓
Registry
↓
Network

78. Проверка цифровой подписи

Для любого:

- EXE;
- DLL;
- SYS;
- MSI;
- OCX;

- CPL.

Проверяем:

```
Signed?  
Valid?  
Trusted?  
Publisher?  
Certificate?  
Timestamp?  
Chain?  
Revoked?  
Expired?
```

79. Hashing

Автоматически:

```
SHA-256  
SHA-1  
MD5
```

для suspicious / selected files.

И можно создать локальную базу:

```
path  
hash  
signature  
publisher  
first_seen  
last_seen
```

80. System Baseline

И вот здесь, на мой взгляд, появляется **главная идея** нашего приложения.

Создать:

```
Windows Baseline
```

и затем проверять:

Current State



Baseline



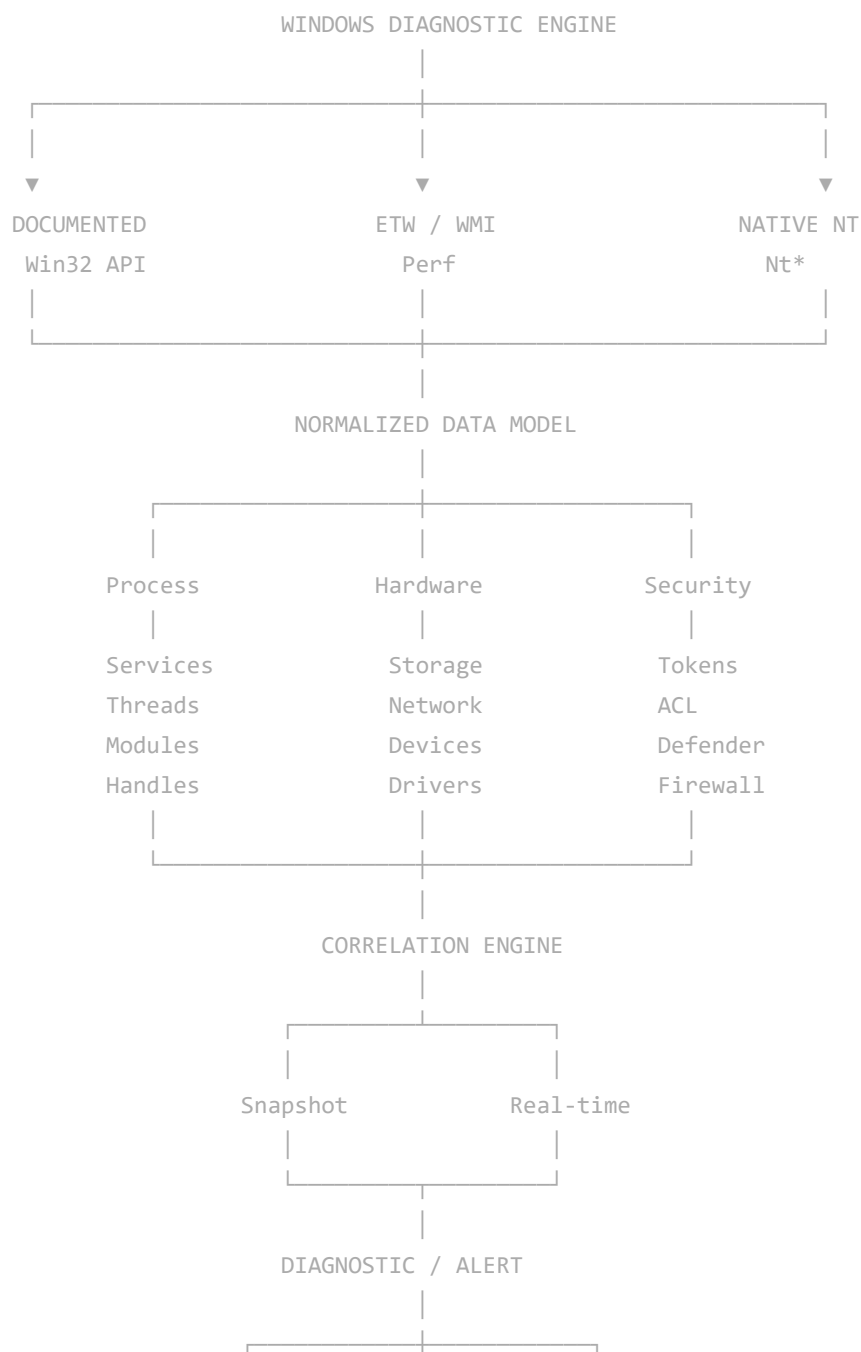
Changes



Anomalies

81. Итого: я бы построил не Process Explorer

А вот такую систему:



И принципиально важно: **не надо выбирать между documented и undocumented API**. Я бы сделал backend с capability levels:

```
LEVEL 1  – Documented Win32
LEVEL 2  – WMI / CIM / Performance Counters
LEVEL 3  – ETW
LEVEL 4  – Native NT API
LEVEL 5  – Undocumented structures
LEVEL 6  – Optional kernel driver
```

Тогда программа на конкретной Windows-машине сама определяет:

```
Process information      ✓
Thread information      ✓
Module enumeration      ✓
Handle enumeration      ✓
ETW                     ✓
WMI                     ✓
Kernel information      ✓
...
```

и **не падает**, если какая-то внутренняя структура изменилась.

Это особенно важно для Native API: Microsoft прямо предупреждает, что `NtQueryInformationProcess` и `NtQuerySystemInformation` могут изменяться или исчезнуть, и рекомендует динамическое получение функций через `LoadLibrary / GetProcAddress` .  Microsoft Lea... +1

Следующий шаг я бы сделал очень конкретным

Из этого списка можно составить **огромную таблицу примерно на 200–300 проверок**:

ID	Подсистема	Проверка	API/DLL	Documented/Native/Undocumented	Требует Admin	Real-time	Надёжность	Windows 10	Windows 11	Результат
----	------------	----------	---------	--------------------------------	---------------	-----------	------------	------------	------------	-----------

И уже по ней проектировать Python-пакет:

```
src/windows/
  api/
    kernel32.py
    advapi32.py
    psapi.py
    ntdll.py
```

```
iphlpapi.py
setupapi.py
wevtapi.py
pdh.py
...
process/
memory/
threads/
modules/
handles/
services/
drivers/
registry/
events/
etw/
network/
devices/
security/
storage/
power/
diagnostics/
```

Это будет гораздо правильнее, чем начинать сразу писать UI. Сначала составим полный каталог Windows capabilities и API, а потом из него построим движок.



ChatGPT can make mistakes. Check important info.